

Web Scraping — Project Report

CodeAlpha Internship | Data Analytics Track | Task 1

About This Task

This task is about collecting data from a website automatically using Python, instead of copying it manually. This process is called **web scraping**.

What I Did

I built a Python script that visits a website, reads its content, and extracts useful information from it — specifically quotes, their authors, and related tags.

How I Did It

- I used two Python libraries: **Requests** to open and read the website's content, and **BeautifulSoup** to find and extract specific pieces of information from the page (like quote text and author name).
- The website I scraped contains multiple pages of quotes. My script automatically moved from one page to the next and collected data from all of them.
- Once all the data was collected, I used **Pandas** to organize it into a clean, structured table.
- Finally, I saved this table as a **CSV file**, which can be opened in Excel or used for further analysis.

What I Learned

- How websites are structured using HTML tags and classes.
- How to identify exactly where the required data is located on a webpage.
- How to handle multiple pages automatically (pagination) instead of doing it manually one by one.
- How to organize and save scraped data in a clean, usable format.
- The importance of being respectful while scraping (adding delays between requests, checking if a website allows scraping).

Tools & Libraries Used

Tool	Purpose
Python	Programming language used
Requests	To fetch webpage content
BeautifulSoup	To parse and extract data from HTML
Pandas	To organize and save data as CSV

Files in This Repository

- **web_scraper.py** — Python code used for scraping
- **scraped_quotes.csv** — Final collected data (quotes, authors, tags)

Submitted as part of the CodeAlpha Data Analytics Internship.